version of OpenCV, then you can install it from the Ubuntu software repositories with a simple *sudo apt-get install libopencv-dev*.

Approach 2: If you want the latest version of OpenCV, you need to build from source; you will have to install the dependencies prior to that. First, to make sure your system is up to date, run the following command:

```
sudo apt-get update
sudo apt-get upgrade
```

Proceed to installing the dependencies, which are segregated as follows.

Essentials: These are libraries and tools required by OpenCV:

```
sudo apt-get install build-essential checkinstall cmake pkg-config
yasm
```

Image I/O: These are libraries for reading and writing various image types. If you do not install them, then the versions supplied by OpenCV will be used:

```
sudo apt-get install libtiff4-dev libjpeg-dev libjasper-dev
```

Video I/O: You need some or all of these packages to add video capturing/encoding/decoding capabilities to the *highgui* module:

```
sudo apt-get install libavcodec-dev libavformat-dev libswscale-
dev libdc1394-22-dev libxine-dev libgstreamer0.10-dev
libgstreamer-plugins-base0.10-dev libv4l-dev
```

Other dependencies:

```
sudo apt-get install checkinstall gir1.2-gst-plugins-base-0.10
gir1.2-gstreamer-0.10 libgstreamer-plugins-base0.10-dev
libgstreamer0.10-dev libslang2-dev libxine-dev libxine1-bin
libxml2-dev
```

Python: Packages needed to build the Python wrappers:

```
sudo apt-get install python-dev python-numpy
```

Other third-party libraries: Install Intel TBB to enable parallel code in OpenCV:

```
sudo apt-get install libtbb-dev
```

GUI: The default GUI for *highgui* in Linux is GTK. You can optionally install QT instead of GTK, and later enable it in the configuration:

```
sudo apt-get install libqt4-dev libgtk2.0-dev
```

Now download OpenCV from *http://bit.ly/WcVc5K* and extract the downloaded file to your home folder. Navigate to the extracted folder, make a sub-folder *'build '* and cd to it; then run the following command:

```
sudo apt-get install cmake-gui
cmake-gui
```

Now provide the source folder, and in the binary folder option, provide the *'build '* folder path. Click *Configure* and select the *'OPENNI '* boxes to include the Kinect support for OpenCV, and click *Configure* again to update. Once you are sure, click *Generate*. Next, compile OpenCV with the usual *make*, and install it with *sudo make install*.

Edit the */etc/ld.so.conf.d/opencv.conf* file and add */usr/local/lib* to it, then run *sudo ldconfig*. Edit the bash.rc file to add the following:

```
PKG_CONFIG_PATH=$PKG_CONFIG_PATH:/usr/local/lib/pkgconfig
export PKG_CONFIG_PATH
```

Now, log out and log back in, or restart the system.

To configure OpenCV with CodeBlocks, go to *Project > Build Options;* then go to *Compiler > Other Settings* and add *`pkg-config --cflags opencv`*. Next, go to the *Linker* settings and in other settings, add: *`pkg-config --libs opencv`*

Note: To run your code from the terminal, use the following command-line pattern:

```
g++ `pkg-config --cflags --libs opencv` -o <name for executable
file> <name of program.cpp>
```

Uninstalling OpenCV completely

If you need to remove your OpenCV installation and return to the pre-installation state, first go to the *build* folder (in the OpenCV folder), and run *sudo make uninstall*. Then delete the entire OpenCV folder. Next, run the following command:

```
sudo find / -name "*opencv*" -exec rm -i {} \;
```

Note: The above command will delete every file with 'opencv ' in its name!

Edit the */etc/ld.so.conf.d/opencv.conf* file and remove */usr/local/lib* from it, then run *sudo ldconfig*. Edit the *bash.rc* file and remove the lines we added to it (see the install section).

A few examples

Example 1: Display RGB data from Kinect in OpenCV (see Figure 1):

```
//Program to display rgb data in OpenCV 2.4.2 using Kinect
```